Response to Reviewer Comments

Contents

1 Response to Reviewer #1 **2**
- 1.1 Comment 1.1 and 1.6— Anomaly Backtracking Logic . . . 2
- 1.2 Comment 1.2 and 1.7 — Temperature Settings . . . 5
- 1.3 Comment 1.3 — Multi-Agent Interaction Latency vs. Manual Modeling . . . 8
- 1.4 Comment 1.4 — BA-Agent User Study Data . . . 8
- 1.5 Comment 1.5 — ECR paper Citation . . . 11
- 1.6 Comment 1.8 — Terminology Consistency . . . 11

2 Response to Reviewer #2 **13**
- 2.1 Comment 2.A — Precise Definition of "Accuracy" . . . 13
- 2.2 Comment 2.B — Same LLM Backbone for All Baselines . . . 13
- 2.3 Comment 2.C — Equation (1) Probability Notation . . . 14
- 2.4 Comment 2.D — Efficiency Metrics by Problem Size . . . 15
- 2.5 Comment 2.E — Human Baseline for Modeling Time **[MERGE with 1.3]** . . . 17
- 2.6 Comment 2.F — Clarify Hardware and Network Latency . . . 18
- 2.7 Comment 2.G — Qualitative Metric for BA-Agent . . . 18

3 Response to Reviewer #3 **20**
- 3.1 Comment 3.1 — `top_p` and `max_tokens` Undefined . . . 20
- 3.2 Comment 3.2 and 3.7 — Dual Use of Variable i . . . 20
- 3.3 Comment 3.3 — Equation (1) Symbols Undefined **[COMPLETED]** . . . 22
- 3.4 Comment 3.4, 3.5 and 3.6 — Variable Inconsistencies in Figure 1 . . . 23
- 3.5 Comment 3.8 and 3.9 — Figure 2 Shows Wrong Problem (Airplane Landing, not ECR) . . . 24
- 3.6 Comment 3.10 — Accuracy Increase Value Inconsistency . . . 27
- 3.7 Comment 3.11 — `is_backward` Formatting Inconsistency . . . 28
- 3.8 Comment 3.12 — Symbols "+" and "-" Defined but Unused . . . 28
- 3.9 Comment 3.13 — S_i^t Variable t Undefined . . . 28
- 3.10 Comment 3.14 — ECR Case Solution Lacks Numerical Evidence . . . 29
- 3.11 Comment 3.16 — Future Research Directions in Conclusion . . . 30
- 3.12 Comment 3.17 — Move Math Model from Appendix to Main Text . . . 31

1 Response to Reviewer #1

1.1 Comment 1.1 and 1.6— Anomaly Backtracking Logic

1.1.1 Comment

1. **Comment 1.1**"The framework claims to reduce 'hallucinations,' but it does not clearly explain how the 'Anomaly Backtracking' logic works in the architecture section."

2. **Comment 1.6**"The paper should include a clear, step-by-step description of how the PM-Agent detects a failure in the TE-Agent and triggers a state transition back to either the ME-Agent or the PY-Agent."

1.1.2 Response

We thank the reviewer for pointing out this ambiguity. We agree that the original manuscript did not make the failure detection and backtracking procedure sufficiently explicit. We have therefore revised Section 3.2 to distinguish (i) failure detection during SOP-MAS-orchestrated execution and verification in Step 6 from (ii) PM-Agent-led diagnosis and stack-based backtracking in Step 7. In the SOP-MAS framework, exception backtracking is triggered whenever the post-generation execution or verification stage fails and `is_backward=True`; therefore, its activation does not depend on whether the TE-Agent has been selected in the current forward pass.

More specifically, when the forward pass includes the TE-Agent, the TE-Agent performs execution and testing step during the forward pass within a sandbox environment and provides structured diagnostic evidence, such as execution results and solver logs, which helps localize the error source more precisely during exception backtracking. After the forward pass, the framework still enters the same unified execution/verification stage; it first checks whether the TE-Agent has already produced a successful execution result and reuses that result if available, otherwise it executes the latest solver code itself. When K is smaller and the TE-Agent is not selected, the framework can still trigger stack-based exception backtracking on the basis of the failure signal returned by the execution and verification stage. In that case, previously selected agents are queried in reverse order to determine whether their outputs caused the failure and to refine them if necessary, although the available error information is less detailed than in the TE-Agent-assisted case.

We further clarify that the transition to the PY-Agent or ME-Agent is not implemented as a hard-coded branch on TE-Agent error types. Instead, the PM-Agent queries agents in reverse workflow order, so code-level defects are typically localized first to the PY-Agent, whereas failures that persist beyond code correction propagate further to the ME-Agent for model-level revision. We have revised the manuscript accordingly to make this trigger procedure explicit and to clarify that the TE-Agent enhances error diagnosis, rather than serving as the sole trigger of exception backtracking.

For clarity, we now restate the TE-Agent and backtracking procedure in list form through two examples built on the same three-port ECR setting described in Section 3.4 of the revised manuscript.

Both examples share the same setting: the DE-Agent extracts S=$[100, 50, 0]$, D=$[0, 0, 150]$, and the transportation cost matrix; the ME-Agent specifies the intended LP formulation with flow-balance constraints at every port; and the PY-Agent translates the formulation into solver code, but accidentally omits the flow-balance constraint for port C during code generation. The two examples then differ only in whether the TE-Agent is selected, so as to isolate its contribution to diagnosis and backtracking.

1. **Example 1 (with TE-Agent): DE-Agent $\rightarrow$ ME-Agent $\rightarrow$ PY-Agent $\rightarrow$ TE-Agent.**

- **Step 5: Iterative modeling.** After the PY-Agent posts the defective code, the TE-Agent performs one supplementary execution within a sandbox environment and returns a structured diagnostic report. A representative excerpt is shown below.

```
{
  "execution_status": "success",
  "diagnostic_summary": "The solver returns an optimal plan, but the
      shipment into port C falls short of the demand D[C]=150.",
  "error_analysis": "Code-level translation defect: the flow-balance
      constraint at port C, present in the ME-Agent formulation, is missing
      from the PY-Agent code.",
  "solver_log_summary": "Solver terminates normally; the failure is not a
      runtime exception but an unenforced constraint.",
  "recommendations": "Re-examine the PY-Agent implementation of the flow-
      balance constraints, especially for the demand node C."
}
```

- **Step 6: Execution and verification.** The framework reuses the TE-Agent execution result and applies the correctness gate. The returned objective and shipment plan deviate from the benchmark target, so the gate emits a failure signal even though execution itself completed.
- **Step 7: Exception backtracking.** With `is_backward=True`, the PM-Agent pops the stack in reverse order, guided by both the framework-level failure signal and the TE-Agent report. Because the report localizes the defect to the code-to-formulation translation, the PM-Agent first queries the PY-Agent for code-level revision; backtracking propagates further to the ME-Agent only if the repaired code still fails verification.

2. **Example 2 (without TE-Agent): DE-Agent → ME-Agent → PY-Agent.**

- **Step 5: Iterative modeling.** The forward pass ends at the PY-Agent and no diagnostic report is produced.
- **Step 6: Execution and verification.** The framework itself executes the latest solver code and applies the correctness gate. As in Example 1, the gate detects that the returned solution is inconsistent with the benchmark target, but the only signal available to downstream backtracking is the framework-level verification failure, without an attribution to the code level or the model level.
- **Step 7: Exception backtracking.** With `is_backward=True`, the PM-Agent still activates backtracking and pops the stack in reverse order. Lacking the TE-Agent report, diagnosis defaults to the monotonic workflow order: the PY-Agent is queried first for code refinement, and the ME-Agent is queried next if the corrected code still fails. Recovery is therefore possible but typically requires more rounds than in Example 1.

Together, the two examples illustrate that exception backtracking is governed by the execution and verification outcome rather than by TE-Agent selection, while the TE-Agent, when available, supplies finer-grained evidence that sharpens fault localization and reduces unnecessary backtracking rounds.

1.1.3 Modifications

1. **Section 3.1, Agent Definition (L277) — MODIFY**

Before: "*The TE-Agent … carries out the execution of the code within a sandbox environment … and subsequently provide a report on the execution.*"

After: "*The TE-Agent was redefined as a supplementary testing and fine-grained diagnosis module. When selected, it performs one supplementary execution during the forward pass and contributes structured diagnostic artifacts such as execution summaries, error analyses, and solver logs to support failure localization during backtracking. SOP-MAS-orchestrated execution and verification nevertheless remain framework responsibilities after the forward pass.*"

2. **Section 3.2, Step 6–7 (L325–335) — MODIFY**

   Before: "*If the TE-Agent is activated, it executes the code generated by the PY-Agent … Only when both automated checks and expert sign-off are satisfied does the workflow advance to Step 7. Otherwise, proceed to Step 7 for exception backtracking. When an execution error is detected and* `is_backward` = `True`*, trigger the backtracking mechanism.*"

   After: "*After the K-step forward pass, the framework enters the unified execution and verification stage. If the TE-Agent is activated, it has already performed one supplementary execution during the forward pass and produced a structured runtime report. The framework therefore first checks whether that TE-Agent execution succeeded; if so, the successful result is reused directly, whereas otherwise the framework executes the latest solver code itself and performs the verification check. If the execution and verification stage succeeds, the workflow proceeds directly to Step 8. Otherwise, if* `is_backward` = `True`*, the PM-Agent triggers Step 7 for exception backtracking; if* `is_backward` = `False`*, the workflow skips backtracking and terminates with the current failure outcome. When backtracking is triggered, the PM-Agent pops the stack in reverse order and queries previously selected agents one by one; if the TE-Agent was selected, its structured execution report is used as part of the diagnostic evidence, whereas otherwise the diagnosis relies on the failure signal returned by the execution and verification stage. Because the forward workflow follows the monotonic order DE-Agent → ME-Agent → PY-Agent → TE-Agent, implementation-level defects are typically examined first at the PY-Agent, whereas failures that persist beyond code correction are propagated further to the ME-Agent for model-level revision.*"

3. **Section 3.3, Item 3 (L351) — MODIFY**

   Before: "*Upon the occurrence of an exception during code execution within the sandbox environment, the TE-Agent retains the error information in the shared message pool.*"

   After: "*Upon the occurrence of an execution or verification failure, the SOP-MAS framework generates the corresponding failure information for backtracking. When selected, the TE-Agent additionally contributes a structured execution report in the shared message pool containing finer-grained diagnostic evidence.*"

4. **Section 4.2, Group 4 description and discussion (L460, L484) — MODIFY**

   Before: "*However, the exception backtracking mechanism remains active, enabling each agent to perform self-checks on its output to detect probable errors. … Group 4 indicates that the exclusion of the TE-Agent still results in a relatively high accuracy, as the testing agent is capable of providing detailed error*

information, and errors can be rectified via the exception-backtracking mechanism."

After: "*However, the exception backtracking mechanism remains active because it is triggered by SOP-MAS execution or verification failure rather than by TE-Agent selection itself. In this setting, previously selected agents are queried in reverse order on the basis of the failure signal returned by the execution and verification stage, without the structured diagnostic report normally provided by the TE-Agent. … Group 4 still achieves a relatively high accuracy, but it remains below the control group. This result suggests that exception backtracking preserves substantial recovery capability even when the TE-Agent is excluded, while the TE-Agent provides more detailed runtime evidence for diagnosing the source of failure.*"

1.2 Comment 1.2 and 1.7 — Temperature Settings

1.2.1 Comment

This comment overlaps substantially with Comment 1.2 and is therefore addressed jointly there.

1. **Comment 1.2:** "The framework relies on specific large language models (LLMs), like GPT-4, but it does not explore how changing temperature settings (t_i) affects its performance."

2. **Comment 1.7:** "The paper should add an experiment that changes the LLM temperature (t_i) and `top_p` values to show how stable the generated Mixed Integer Programming (MIP) formulations are."

1.2.2 Response

We thank the reviewer for this constructive and inspiring suggestion. We wish to clarify that Section 4.2 already presents a comprehensive sensitivity analysis examining the effect of temperature on framework performance. Specifically, a grid search was performed over $t_{ME}, t_{PY} \in [0.2, 0.4, 0.8, 1.0]$ while varying $K \in [2, 5]$, using AR as the evaluation metric. The results, presented in Figure 3, demonstrate that lower temperatures consistently yield higher accuracy for both the ME-Agent and PY-Agent.

Regarding the scope of temperature coverage, we focused on t_{ME} and t_{PY} because these two agents are directly responsible for mathematical model construction and executable code generation, respectively, which are the primary determinants of solution accuracy. The temperatures of the remaining agents have a relatively minor impact for different reasons: the DE-Agent performs structured data extraction whose output format is largely determined by the input schema; the PM-Agent's selection strategy is constrained by the sub-sequence rules, which limits the effect of temperature-induced randomness; the TE-Agent is not a primary determinant of formulation accuracy, although it can indirectly improve final performance by providing finer-grained diagnostic evidence during backtracking; and the BA-Agent operates after optimization and therefore does not materially affect model accuracy, as validated by the ablation study in Section 4.3 (Group 2 and Group 4).

Concerning the `top_p` parameter raised jointly in Comment 1.7, it is kept at its default value of 1.0 throughout the main experiments. To directly address the reviewer's request for empirical evidence rather than relying on the design rationale alone, we conducted a dedicated `top_p` sensitivity study on the SOP-MAS framework with the DeepSeek-V3 backbone. Specifically, we performed a 4×2 factorial sweep over `top_p` $\in \{0.5, 0.7, 0.9, 1.0\}$ and $t_{ME} = t_{PY} \in \{0.2, 0.8\}$, applying the swept values only to the ME-Agent and PY-Agent (other agents remain deterministic). The study covers five representative instances spanning ComplexOR (`aircraft_landing`,

`diet_problem`, `knapsack_optimization`) and NLP4LP (instances `74` and `187`), each repeated R=3 times under `is_backward` = `False`. Beyond AR, we additionally report an Objective-Agreement Rate (OAR), defined per instance as the fraction of repeat pairs whose objective values agree within a relative tolerance of 10^{-4} and then averaged across instances; OAR captures stability independently of correctness. The results in Table 1 show that AR is essentially invariant to `top_p` (0.600 in seven of the eight configurations), and OAR remains 1.000 except in the highest-stochasticity corner (`top_p`=0.9, T=0.8), where it drops modestly to 0.733. This empirically confirms that, for this task, varying `top_p` does not alter the conclusions drawn from the temperature-only sweep in Section 4.2, consistent with the OpenAI API documentation [1] guidance that practitioners alter `top_p` *or* `temperature` but not both, and with prior work such as Chain-of-Experts (CoE) [Xiao et al., 2023] where only temperature is tuned.

Table 1: `top_p` sensitivity ablation on the SOP-MAS framework (DeepSeek-V3 backbone, K=5, `is_backward` = `False`, 5 instances × 3 repeats = 15 runs per configuration). AR is the Accuracy Rate; OAR is the Objective-Agreement Rate (averaged pairwise consistency of objective values across repeats, per instance).

Config	`top_p`	t_{ME}=t_{PY}	n_{runs}	AR	OAR
c1	0.5	0.2	15	0.600	1.000
c2	0.7	0.2	15	0.600	1.000
c3	0.9	0.2	15	0.600	1.000
c4	1.0	0.2	15	0.600	1.000
c5	0.5	0.8	15	0.600	1.000
c6	0.7	0.8	15	0.600	1.000
c7	0.9	0.8	15	0.533	0.733
c8	1.0	0.8	15	0.600	1.000

Further, we consulted the official OpenAI API documentation (reproduced in Figure 1), which recommends adjusting `temperature` *or* `top_p` but not both. The empirical evidence in Table 1 corroborates this for the present task.

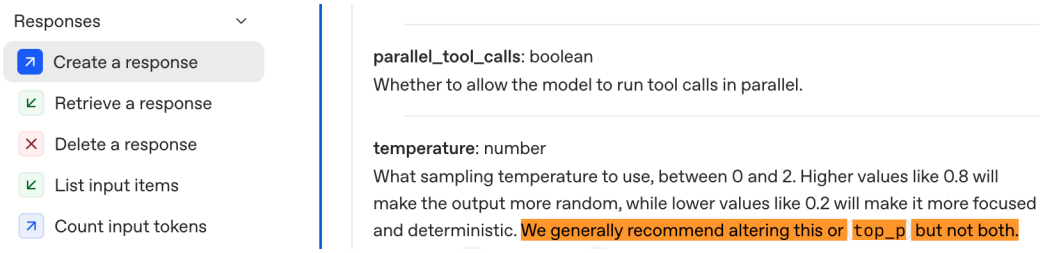

Figure 1: Excerpt from the OpenAI API documentation for Chat Completions, showing the official recommendation that practitioners alter `temperature` *or* `top_p` but not both. This documentation guidance is corroborated by the empirical results in Table 1.

Regarding MIP formulation stability across runs, we acknowledge that variable naming and symbolic representation inevitably differ across runs due to the inherent stochasticity of LLM generation. However, such differences are cosmetic and do not affect mathematical equivalence. For structural consistency: when exception backtracking is enabled (`is_backward` = `True`), the verification–backtracking loop corrects formulation errors based on ground-truth comparison, so the final formulation reliably converges to a correct model. When backtracking is disabled (`is_backward` = `False`), structural inconsistency across runs is possible, a model may use a

[1] https://platform.openai.com/docs/api-reference/chat/create

different constraint set or modeling approach while still yielding the correct optimal objective value. The sensitivity analysis in Section 4.2 was conducted under `is_backward` = `False` to isolate the effect of temperature; the resulting AR values therefore reflect single-pass generation reliability rather than post-correction consistency.

We have revised Section 4.2 to (1) explicitly justify the focus on t_{ME} and t_{PY}, (2) report the dedicated `top_p` sensitivity ablation summarized in Table 1, which empirically shows AR and OAR are essentially invariant to `top_p` across the four sampled values, and (3) correct an inconsistency in Step 6 where the verification criterion allowed a Feasible status, which has been tightened to require an Optimal status to align with the AR definition in Section 4.1.

1.2.3 Modifications

1. **Section 4.2, Sensitivity Analysis (L430) — MODIFY (restructure and merge two paragraphs into one)**

   Before: "*Within the SOP-MAS framework, pivotal parameters encompass the maximum collaboration count K of the PM-Agent, alongside the temperatures of the ME-Agent and PY-Agent, denoted as t_{ME} and t_{PY}, respectively. The parameter K exerts influence over the overall depth and breadth of the workflow by modulating the number of agent invocations. Temperature significantly affects the optimization modeling and solution outputs generated by the agents. Specifically, elevated temperatures foster the development of more innovative solutions, while reduced temperatures bolster deterministic reasoning, and in consequence ensuring stability and compliance of the outputs. This trade-off between precision and creativity mandates meticulous calibration of the temperature parameters tailored to specific task scenarios. [paragraph break] Among all agents, the temperatures of the ME-Agent and PY-Agent are prioritized for sensitivity analysis because…*"

   After: "*Within the SOP-MAS framework, the maximum collaboration count K of the PM-Agent governs the depth and breadth of the workflow by modulating the number of agent invocations. Among the six agents, the temperatures of the ME-Agent and PY-Agent, denoted as t_{ME} and t_{PY}, are the primary determinants of solution accuracy, because these two agents are directly responsible for mathematical model construction and executable code generation, respectively. The temperatures of the remaining agents (DE-Agent, PM-Agent, TE-Agent, and BA-Agent) have a relatively minor impact on the optimization outcome, as validated by the ablation study in Section 4.3. Temperature significantly affects the optimization modeling and solution outputs generated by the agents: elevated temperatures foster more innovative solutions, while reduced temperatures bolster deterministic reasoning, ensuring stability and compliance of the outputs. This trade-off between precision and creativity mandates meticulous calibration of the temperature parameters tailored to specific task scenarios.*"

2. **Section 3.2, Step 6 (L315) — MODIFY**

   Before: "*The solution is deemed correct only if (i) the solver status is Optimal or Feasible, and (ii) the objective value matches the ground-truth optimum.*"

   After: "*The solution is deemed correct only if (i) the solver reports an Optimal status, and (ii) the objective value matches the ground-truth optimum.*"

1.3 Comment 1.3 — Multi-Agent Interaction Latency vs. Manual Modeling

1.3.1 Comment

"There is not enough discussion about the delay caused by the multi-agent interaction layers compared to traditional manual modeling timelines."

1.3.2 Response

We deeply appreciate the reviewer's rigorous attention to detail. We agree that the manuscript should more clearly distinguish the measured automated latency of SOP-MAS from the broader expert-centered modeling cycle. In response, we revised the manuscript to discuss this distinction more explicitly while retaining the workflow-level acceleration claim.

Specifically, we now clarify that the reported 48.75 s is the end-to-end automated latency of SOP-MAS, which already includes multi-agent interaction, API communication, and local execution. Therefore, the delay introduced by the interaction layers is not omitted from the reported runtime, but is already part of the measured latency. At the same time, we clarify the scope of the comparison: the reported 48.75 s refers to the automated generation of an executable initial model, rather than the full human modeling cycle. For complex real-world ECR problems, manual formulation typically involves repeated requirement clarification, data checking, model revision, code debugging, and stakeholder validation, and therefore often spans hours or even weeks in practice. In this sense, SOP-MAS compresses the routine formulation stage to sub-minute automated generation and shifts the remaining work to expert review, validation, and refinement, yielding an hour-scale human-in-the-loop workflow rather than a prolonged expert-centered process.

1.3.3 Modifications

1. **Section 4.2 (L426) — MODIFY**

   Before: "*Its primary objective is to accelerate the expert modeling workflow, from weeks to hours, by automating routine tasks such as data parsing, constraint drafting, and code generation, while simultaneously surfacing potential inconsistencies or violations for expert review.*"

   After: "*Its primary objective is to accelerate the expert modeling workflow, which in practice often spans hours or even weeks for complex real-world problems, by automating routine tasks such as data parsing, constraint drafting, and code generation, while simultaneously surfacing potential inconsistencies or violations for expert review. Final validation, business-rule checking, and strategic acceptance remain the responsibility of human experts.*"

2. **Section 4.4 (L508) — ADD** ·

   After: "*This latency includes multi-agent interaction, API communication, and local execution, and should be interpreted as the time required to generate an executable initial model rather than the full human modeling cycle, which also involves requirement clarification, iterative formulation revision, debugging, and stakeholder validation.*"

1.4 Comment 1.4 — BA-Agent User Study Data

1.4.1 Comment

"The claim that the system reduces barriers for non-experts lacks user-study data to support the effectiveness of the 'Business Analyst Agent.' "

1.4.2 Response

We thank the reviewer for this insightful observation. We agree that the BA-Agent should be supported by evidence beyond Accuracy Rate (AR), because it does not alter the mathematical correctness of the generated model but instead translates solver outputs into business-accessible interpretations for non-expert stakeholders.

Although a formal user study remains beyond the scope of the present revision, we have strengthened the manuscript with two concrete pieces of evidence from actual BA-Agent outputs. First, in the illustrative three-port ECR example (Section 3.4), the BA-Agent correctly interprets the optimal objective value of CNY 650 and explains why the selected flows (x_{AB}=100, x_{BC}=150) satisfy demand while the direct A–C route and reverse movements remain zero. This serves as a transparent sanity check for interpretation accuracy. For direct inspection, we reproduce below a translated excerpt from the actual BA-Agent output generated for this example:

Representative BA-Agent output (translated from the actual run).

```
{
  "executive_summary": "The optimizer found the minimum-cost plan
  for the inter-port empty-container repositioning task, with a
  total transportation cost of CNY 650. The key insight is that
  routing through the low-cost A->B and B->C arcs minimizes cost
  while fully satisfying Port C's demand.",
  "cost_breakdown": {
    "total_cost": "650.0",
    "cost_items": [
      {"category": "A->B repositioning cost", "amount": "200.0"},
      {"category": "B->C repositioning cost", "amount": "450.0"}
    ]
  },
  "key_finding": "Port B acts as the core hub of the repositioning
  network.",
  "actionable_recommendation": "Immediately arrange the shipment of
  100 empty containers from Port A to Port B and 150 empty
  containers from Port B to Port C."
}
```

Second, in the large-scale empirical case (Section 5.3), the BA-Agent summarizes the CNY 1,355,388 solution into an exact cost decomposition, repositioning CNY 925,238 (68.26%), leasing CNY 406,000 (29.95%), and holding CNY 24,150 (1.78%),and further provides actionable operational recommendations. These additions demonstrate that the BA-Agent does more than paraphrase variable values: it converts mathematically valid solutions into structured managerial insight.

To clarify this point more precisely, we also revised the ablation discussion (Section 4.3) in a more conservative manner. Rather than embedding a formal qualitative framework in the manuscript, we now state that AR has limited ability to capture the BA-Agent's downstream interpretive role because the BA-Agent operates after solution generation, and we point readers to the concrete demonstrations in Sections 3.4 and 5.3.

Although we cannot conduct this study within the current revision, we have given careful thought to its design. The core idea is to invite domain experts (e.g., port operations managers and supply-chain modeling experts) to independently score each BA-Agent report on three dimensions (interpretation accuracy, completeness, and actionability) using structured scoring instruments, and then compute inter-rater agreement to obtain a reliable, quantitative measure of interpretive quality. We plan to carry out this evaluation as a natural extension of the present work.

1.4.3 Modifications

1. **Section 3.4 (L407) — MODIFY**

   Before: "*Fifth, the BA-Agent translates the optimal solution into a business-analysis report, identifying that the strategy routes surplus containers to the deficit port via the most cost-effective arcs.*"

   After: "*Fifth, the BA-Agent translates the optimal solution (x_{AB}=100, x_{BC}=150, with all other flows equal to zero) into a business-analysis report, correctly stating the optimal total transportation cost of CNY 650, explaining why the low-cost A–B and B–C arcs are selected to satisfy port C's demand through Port B as the transfer hub, and clarifying why the expensive direct A–C route and reverse movements remain zero. This small instance therefore provides a transparent sanity check for the interpretation accuracy of the BA-Agent.*"

2. **Section 4.3 (L522) — MODIFY**

   Before: "*In practice, the BA-Agent contributes value on a qualitative dimension that is orthogonal to AR. By translating raw solver output into a structured business analysis report—comprising cost attribution analysis, key operational findings, and actionable recommendations—it substantially lowers the interpretive barrier for non-expert stakeholders. As demonstrated in Section 5.3, the BA-Agent's output converts complex numerical results into business-accessible insights such as cost driver identification and what-if scenario implications. This contribution aligns with emerging work on LLM-driven optimization result interpretation [Li et al., 2023], where the value lies not in improving solution quality but in enhancing its communicability and usability for business decision-making.*"

   After: "*Because the BA-Agent operates after solution generation, removing it has limited effect on AR even though it reduces the interpretability of the final output. Its main contribution is to translate solver outputs into interpretable business reports for end users, as illustrated by the three-port example in Section 3.4 and the empirical case in Section 5.3, where it explains routing decisions, decomposes total cost, and provides managerial recommendations. A formal qualitative evaluation of this interpretive contribution is left for future work.*"

3. **Section 5.3 (after the cost-comparison table) — ADD**

   After: "*Beyond the numerical comparison in the cost-comparison table, the BA-Agent converts the large-scale solver output for Scenario 2 into a structured business report. Specifically, it identifies the total cost as CNY 1,355,388 and attributes 68.26% to repositioning (CNY 925,238), 29.95% to leasing (CNY 406,000), and 1.78% to holding (CNY 24,150). It further highlights the dominant supply and receiving hubs, identifies shortage-prone inland nodes, and recommends prioritizing the Yingkou–Dalian repositioning corridor, securing long-term leasing agreements in inland shortage locations, and clearing stagnant inventories in peripheral nodes. This example illustrates how the BA-Agent transforms a mathematically optimal solution into an actionable managerial interpretation.*"

4. **Section Conclusions (line 597) — MODIFY**

   After: "*Finally, conducting a multi-rater user study with logistics practitioners to evaluate the BA-Agent's output framework comprising interpretation accu-*

racy, completeness, and actionability sub-dimensions) would provide rigorous empirical evidence of its qualitative contribution."

1.5 Comment 1.5 — ECR paper Citation

1.5.1 Comment

"In Section 2.4 (Challenges). It is suggested to refer from the paper: 'A novel permissioned blockchain approach for scalable and privacy-preserving IoT authentication'. This citation addresses privacy and trust issues in decentralized environments, which is important for discussing the security of data shared between port agents in the ECR network."

1.5.2 Response

We appreciate the reviewer's valuable suggestion. The cited paper is highly relevant to the challenges discussed in Section 2.4, particularly because the decentralized nature of EC networks requires sensitive operational data, such a container inventories and transport schedules, to be shared across organizationa boundaries. We have therefore added this reference an revised the discussion to highlight the importance of privacy-preservin and trust mechanisms for effective multi-stakeholde collaboration.

1.5.3 Modifications

1. **Section 2.4 (L253) — ADD**

   After: "*. This decentralized nature of ECR networks, where sensitive operational data (e.g., container inventories and transport schedules) must be shared across organizational boundaries, underscores the importance of privacy-preserving and trust mechanisms for effective multi-stakeholder collaboration [27]*"

1.6 Comment 1.8 — Terminology Consistency

1.6.1 Comment

"The terminology must be consistent; the terms 'SOP-MAS' and 'LLM-agent system' should be used in the same way throughout."

1.6.2 Response

A comprehensive terminology audit was performed. "LLM-agent system" did not appear in the original text. We have unified the framework reference as **SOP-MAS** throughout the paper: (1) the acronym is formally introduced in the Abstract upon first mention; (2) all subsequent framework references use "SOP-MAS" or "the SOP-MAS framework" instead of informal paraphrases; (3) "SOP-MAS system" redundancy is removed. The generic terms "multi-agent system" and "MAS" are reserved for the general concept in the literature review.

1.6.3 Modifications

1. **Abstract (L172) — MODIFY**

   Before: "*by designing a multi-agent collaborative framework grounded in artificial intelligence large language models. This framework interprets …*"

   After: "*by designing SOP-MAS (Standard Operating Procedure–Multi-Agent*

System), a multi-agent collaborative framework grounded in artificial intelligence large language models. This framework interprets …"

2. **Methodology Section (L267) — MODIFY**

   Before: "*This research proposes a specialized LLM-based MAS framework predicated upon the Standardized Operating Procedure (SOP) paradigm.*"

   After: "*This section details the SOP-MAS framework, which is predicated upon the Standardized Operating Procedure (SOP) paradigm.*"

3. **Section 4.2 (L429) — MODIFY**

   Before: "*Within the framework of SOP-MAS system,*"

   After: "*Within the SOP-MAS framework,*"

2 Response to Reviewer #2

2.1 Comment 2.A — Precise Definition of "Accuracy"

2.1.1 Comment

"Define 'accuracy' precisely. Distinguish between successful code execution and reaching the global optimum (optimality gap). High-impact journals require measuring the mathematical quality of the solution, not just syntax."

2.1.2 Response

We revised the metric definitions into explicit indicator-based formulas. In direct response to the reviewer, we formalized AR to distinguish executable correctness from mathematical correctness. In addition, for completeness and notational consistency across the evaluation metrics, we also expressed CER and RER mathematically. Specifically, an instance is counted as accurate only if the generated code compiles, executes without runtime errors, the solver reports an Optimal status, and the returned objective value matches the ground-truth optimum within a numerical tolerance. This is consistent with the verification criteria already described in Step 6 (Section 3.2).

2.1.3 Modifications

1. **Section 4.1 (L394) — MODIFY**

   Before: "*AR represents the proportion of codes that successfully satisfy all test cases.*"

   After: "*For each test instance $i \in \{1, \ldots, N\}$, let $c_i \in \{0, 1\}$ indicate successful compilation, $r_i \in \{0, 1\}$ indicate execution without runtime errors, s_i denote the solver status, z_i denote the objective value returned by the generated code, and $\hat{z}_i$ denote the ground-truth optimal objective value. CER and RER are defined as*

   $$\mathrm{CER} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(c_i = 0), \qquad \mathrm{RER} = \frac{\sum_{i=1}^{N} \mathbf{1}(c_i = 1,\ r_i = 0)}{\sum_{i=1}^{N} \mathbf{1}(c_i = 1)}.$$

   An instance is considered correctly solved only when the generated code compiles, executes without error, attains an optimal solver status, and yields an objective value within tolerance of the ground truth, i.e.,

   $$a_i = \mathbf{1}\left(c_i = 1,\ r_i = 1,\ s_i = \mathrm{Optimal},\ \frac{|z_i - \hat{z}_i|}{\max(1, |\hat{z}_i|)} \le \varepsilon\right).$$

   The accuracy rate is then defined as

   $$\mathrm{AR} = \frac{1}{N}\sum_{i=1}^{N} a_i.$$

   In this study, $\varepsilon = 10^{-6}$."

2.2 Comment 2.B — Same LLM Backbone for All Baselines

2.2.1 Comment

"Explicitly confirm if all baselines (OptiMUS, CoE, etc.) used the same DeepSeek-v3 backbone. Paradigm effectiveness cannot be validated if the underlying language models differ across comparisons."

2.2.2 Response

We thank the reviewer for raising this important point. We fully agree that a paradigm-level comparison is meaningful only when the underlying LLM is controlled. In our experiments, all methods, including SOP-MAS, SPM, CoT, CoE, and OptiMUS, were run with the same DeepSeek-v3-0324 backbone and under identical hardware conditions. For CoE and OptiMUS, we used the authors' public code repositories (https://github.com/xzymustbexzy/Chain-of-Experts and https://github.com/teshnizi/OptiMUS, respectively) and adapted their LLM interface to DeepSeek-v3-0324; for SPM and CoT, we directly implemented the published prompting procedures on the same backbone. The benchmark instances and evaluation settings were also held fixed across methods, including five independent runs per instance with $K = 5$. Therefore, the reported performance differences reflect the effectiveness of the collaboration paradigm rather than differences in the underlying model. We have clarified this explicitly in Section 4.1.

2.2.3 Modifications

1. **Section 4.1 (L390) — MODIFY**

   Before: "*We adopt DeepSeek-v3-0324 as the backbone LLM, given its leading position among open weight models.*"

   After: "*We adopt DeepSeek-v3-0324 as the backbone LLM for all methods, given its leading position among open weight models.*"

2. **Section 4.1 (L390) — ADD**

   After: "*All baselines were evaluated under the same DeepSeek-v3-0324 backbone and identical hardware conditions to ensure a fair comparison.*"

2.3 Comment 2.C — Equation (1) Probability Notation

2.3.1 Comment

"Re-evaluate Equation 1's probabilistic notation. If agent selection is governed by deterministic prompts rather than actual probability distributions, simplify the math to avoid unnecessary 'math-washing.' "

2.3.2 Response

We thank the reviewer for this careful observation. After re-checking the implementation, we agree that the original probabilistic notation overstated the role of randomness in PM-Agent selection. In the current implementation, the PM-Agent is invoked through an LLM call, but the candidate set is restricted by the sub-sequence rule, the prompt explicitly instructs the model to choose the smallest admissible `agent_id`, and decoding is performed with temperature 0.0. The realized selection process is therefore better described as a deterministic, state-dependent mapping over an ordered admissible set rather than as a stochastic policy. The relevant PM-Agent prompt template is reproduced below:

```
# PROBLEM DESCRIPTION
Current optimization problem to solve: {problem_description}

# SELECTION CRITERIA
1. **STRICT ORDERING REQUIREMENT** (HIGHEST PRIORITY):
   - You MUST select agents in the order they appear in {agent_list}
   - The agent_id sequence MUST be monotonically increasing
   - NEVER skip agents or select out of order unless already selected
```

```
2. Available Options:
   - Already selected: {selected_agent_list}
   - Available pool: {allowed_agent_list}
   - Remaining quota: {remaining_collaborate_nums} / {max_collaborate_nums}

3. Selection Logic:
   - From {allowed_agent_list}, select the agent with the
     SMALLEST agent_id
```

This prompt structure shows that the PM-Agent operates under an explicit ordering constraint over the current admissible set. Accordingly, we revised Equation (1) and the surrounding text in the manuscript to remove the probabilistic formulation and restate the PM-Agent policy as a deterministic selection function conditioned on the current state and admissible agent set. We also updated the notation explanation in Comment 3.3 to match this revision.

2.3.3 Modifications

1. **Section 3.2 Step 2 and Equation (1) (L330–334) — MODIFY**

   Before: "*The policy selection function is defined as:* $\pi_{\mathrm{PM}}(e|s) = Pr\{A_k = e \mid S_k = s\}$. *... The probability arises from the stochastic nature of LLM generation, since the PM-Agent determines the next agent via an LLM call rather than a hard-coded rule.*"

   After: "*The PM-Agent selection function is defined as:* $a_k = \pi_{\mathrm{PM}}(s_k)$, *where* $a_k \in \mathcal{A}_k$. *The selectable set is restricted by the sub-sequence rule and the PM-Agent is decoded deterministically; accordingly, the realized selection process is described as a deterministic, state-dependent mapping over an ordered admissible set.*"

2.4 Comment 2.D — Efficiency Metrics by Problem Size

2.4.1 Comment

"Provide efficiency metrics correlated to problem size. Show how runtime and accuracy scale as the number of ports and time periods increase to prove industrial-grade scalability."

2.4.2 Response

We appreciate the reviewer's concern regarding industrial-grade scalability. After careful analysis of the SOP-MAS workflow, we clarify that the framework's modeling runtime does not scale with problem size. The key insight lies in a deliberate architectural design: **problem description and concrete data are strictly separated**.

In SOP-MAS, the problem description P (e.g., `description.txt`) is expressed in abstract symbolic form, parameters, variables, and constraints defined using index notation (e.g., $i \in I$ for ports, $t \in T$ for time periods), while the concrete numerical data (e.g., port coordinates, demand matrices) resides in separate data files (e.g., `sample.json`). This separation is essential because embedding concrete data directly into the problem description would cause the text length to grow prohibitively with problem scale, overwhelming the LLM's context window and comprehension capacity. To illustrate this point, consider the following comparison:

Case A: Problem description with embedded data (not adopted). If concrete data were embedded, even a simple 3-port ECR example would need to enumerate all values explicitly:

```
# ECR Problem: 3 ports (A, B, C), T=1

The supply vector is S = [100, 50, 0] (TEU), where
  port A supplies 100 TEU, port B supplies 50 TEU, port C supplies 0 TEU.
The demand vector is D = [0, 0, 150] (TEU), where
  port A demands 0 TEU, port B demands 0 TEU, port C demands 150 TEU.
Unit transportation costs (CNY/TEU):
  c(A->B) = 2, c(A->C) = 10, c(B->C) = 3
  c(B->A) = 2, c(C->A) = 10, c(C->B) = 3
Storage cost h_i = 0, initial inventory I_i0 = 0.
  ... (for larger scale: 3 seaports + 5 dry ports + 7 time periods
   => text grows to O(|N|^2 * |T|) entries)
```

Case B: Problem description with separated data (adopted by SOP-MAS). The description uses abstract symbolic notation, remaining concise regardless of scale:

```
# ECR Problem: Multi-period Network Flow Optimization

Network: Directed graph G=(N,E), where N = H (seaports) union L (dry ports).
Edges E represent repositioning routes: seaport-seaport,
seaport-dry port, and dry port-dry port.
Lambda_ij describes the hinterland relationship between seaport i
and dry port j.

Parameters:
- S_i^t: supply of empty containers at node i in period t
- D_i^t: demand at node i in period t
- c_ij: unit transport cost from node i to node j
- h_i: holding cost at node i
- U_i^max: storage capacity at node i
(Data provided separately in sample.json)
```

As shown, Case B's description length is independent of the number of ports or time periods, whereas Case A's text length grows proportionally to $O(|N|^2 \times |T|)$ as problem scale increases. For the Liaoning case with 3 seaports, 5 dry ports, and 7 time periods, Case A would require enumerating hundreds of specific values, whereas Case B's abstract description remains unchanged.

This design has the following implications for each step of the SOP process:

- **Steps 1–5 (Modeling):** Since P remains fixed in length regardless of how many ports or time periods are involved, the DE-Agent's parameter extraction, the ME-Agent's model formulation, and the PY-Agent's code generation all produce identical outputs in terms of model structure and code template. The LLM API call time for these steps is therefore constant.

- **Step 6 (Execution):** After the forward pass, the framework enters a unified execution stage around the generated `optimize(data)` function, where concrete data is loaded from external files (`sample.json`). If the TE-Agent was selected earlier in the forward pass, its successful execution result can be reused directly; otherwise, or if reuse is not possible, the framework executes the latest solver code itself. Only at this stage does the actual problem scale manifest, affecting the optimization solver's (such as Gurobi or OR-Tools) runtime. Increasing the number of ports or time periods increases the number of decision variables and constraints fed to the solver, but the solver code template itself remains unchanged.

- **Scalability implication:** The modeling overhead of SOP-MAS (Steps 1–5) is bounded

by the sub-minute LLM API call time and is independent of problem scale. The computational bottleneck shifts entirely to the solver execution phase, whose scalability depends on the choice of solver (e.g., OR-Tools, Gurobi, CPLEX) and has been extensively validated by the OR community. This separation of concerns allows SOP-MAS to serve as a scale-agnostic modeling layer on top of existing solver infrastructure.

We have added a corresponding discussion to Section 4.4.

2.4.3 Modifications

1. **Section 4.4 (L507) — ADD**

   After: "*The modeling runtime of SOP-MAS is dominated by LLM API calls rather than problem scale. This scale-independence stems from the architectural separation between the abstract symbolic problem description (parameters, variables, constraints) and the concrete numerical data. Because the problem description is expressed in index notation (e.g., $i \in I$, $t \in T$), the model structure generated by the ME-Agent and the code template produced by the PY-Agent remain invariant to the number of ports or time periods. Consequently, the sub-minute modeling overhead of SOP-MAS stays constant as problem size increases. The actual problem scale, manifested in the number of decision variables and constraints, only affects the solver execution phase, whose scalability has been extensively validated by the OR community. This separation of concerns makes SOP-MAS suitable for industrial-scale applications where the computational bottleneck resides in the solver, not in the modeling process.*"

2.5 Comment 2.E — Human Baseline for Modeling Time [MERGE with 1.3]

2.5.1 Comment

"Support 'reduced modeling time' claims with a human baseline. Compare the 48-second SOP-MAS runtime against the actual hours an expert requires to manually model the Liaoning case."

2.5.2 Response

Thank you for this important suggestion. We fully agree that a human baseline is important for interpreting reduced modeling-time claims. And we note that Comment 2.E and Comment 1.3 are closely related, as both concern the interpretation of modeling time reduction claims. To avoid fragmented discussion, we have merged this point into Comment 1.3 and revised the manuscript to explicitly clarify that the reported 48-second runtime reflects the end-to-end automated latency of SOP-MAS for generating an executable initial model, with multi-agent interaction already included in that measured latency, rather than the full human modeling cycle, which also involves requirement clarification, iterative formulation revision, debugging, and stakeholder validation. We further clarify that the comparison is intended as a workflow-level statement: for complex real-world cases such as the Liaoning ECR application, the expert-centered formulation process often spans hours or even weeks in practice, whereas SOP-MAS reduces the routine formulation stage to sub-minute automated generation and shifts the remaining work to expert review and refinement.

2.5.3 Modifications

Merged into Comment 1.3. Click here to jump to the modifications for Comment 1.3.

2.6 Comment 2.F — Clarify Hardware and Network Latency

2.6.1 Comment

"Clarify the Intel i5-4210U hardware usage. If an API was used for DeepSeek-v3, separate network latency from agent reasoning time to ensure the efficiency analysis is meaningful."

2.6.2 Response

We sincerely thank the reviewer for this insightful and constructive comment. This point is highly important and has directly helped us improve the clarity, rigor, and overall quality of the manuscript. We agree that hardware usage and latency composition should be stated explicitly for a meaningful efficiency interpretation. In the revised manuscript, we now clarify that the Intel i5-4210U machine is used for local orchestration and execution tasks (workflow control, message-pool management, and local Gurobi execution), while DeepSeek-v3 is accessed through a cloud API. We further clarify that the reported per-agent runtimes are end-to-end caller-side latencies, where LLM-call durations inherently bundle network round-trip and server-side inference time. Because this study reports practical end-to-end runtime rather than provider-internal service-time telemetry, we do not claim a strict decomposition between network and inference components; however, we make this boundary explicit and discuss it as a limitation to avoid over-interpretation.

2.6.3 Modifications

1. **Section 4 (opening paragraph, now around L415) — MODIFY**

   Before: "*The entirety of the experiments is conducted in Python 3.11.9 and performed on a Windows 10 Professional platform featuring an Intel i5-4210U CPU and 32 GB of DDR3 memory.*"

   After: "*The entirety of the experiments is conducted in Python 3.11.9 and performed on a Windows 10 Professional platform featuring an Intel i5-4210U CPU and 32 GB of DDR3 memory. DeepSeek-v3 serves as the backbone LLM and is accessed via a cloud API; the local machine is responsible solely for orchestrating the multi-agent workflow, executing generated solver code during the verification stage, optionally performing the TE-Agent's supplementary diagnostic execution when that agent is selected, and managing the message pool.*"

2. **Section 4.4 (after L507) — ADD**

   After: "*Furthermore, the per-agent times reported above are end-to-end latencies measured from the caller side, and each LLM-mediated stage inherently bundles network round-trip and server-side inference into one indivisible measurement.*"

2.7 Comment 2.G — Qualitative Metric for BA-Agent

2.7.1 Comment

"Propose a qualitative metric for the Business Analyst agent. Since it does not affect mathematical accuracy, demonstrate its value through user-interpretability scores or decision-making speed improvements."

2.7.2 Response

We sincerely thank the reviewer for this constructive and inspiring suggestion. Proposing a qualitative evaluation framework for the BA-Agent is a highly valuable insight that significantly strengthens the interpretability narrative of our work. We address this concern in conjunction with Comment 1.4. In the revised manuscript, we now clarify more conservatively that the BA-Agent's contribution is primarily interpretive and therefore is not well captured by Accuracy Rate (AR), while Sections 3.4 and 5.3 provide two concrete demonstrations of that contribution. In this response letter, we further operationalize the BA-Agent's qualitative value along the following three dimensions:

1. **Interpretation accuracy** (*Is the interpretation correct?*): Whether the BA-Agent's cost attribution and findings are mathematically consistent with the solver output.

2. **Completeness** (*Is the interpretation thorough?*): Whether the report covers the key cost drivers, operational findings, and actionable recommendations.

3. **Actionability** (*Can a manager act on this?*): Whether the recommendations are specific enough to inform operational decisions.

The BA-Agent's outputs in both the illustrative and empirical cases show how these dimensions could be assessed in practice. However, a rigorous multi-rater user study, e.g., with logistics practitioners scoring the BA-Agent's output on each dimension using structured scoring instruments, constitutes a natural and important extension of this work. We have added a discussion of this direction in the Conclusions (Section 6).

2.7.3 Modifications

Addressed jointly with Comment 1.4; the manuscript revisions appear in Sections 3.4, 4.3, and 5.3, while the three evaluation dimensions are discussed in this response letter and referenced in Section 6 as future work.

3 Response to Reviewer #3

3.1 Comment 3.1 — `top_p` and `max_tokens` Undefined

3.1.1 Comment

"In lines 314-315, the terms top_p and max_tokens are not defined."

3.1.2 Response

We deeply appreciate the reviewer's careful and precise reading. You are correct that `top_p` and `max_tokens` were left undefined, which may confuse readers unfamiliar with LLM inference hyperparameters; we have revised accordingly.

`top_p` (nucleus sampling threshold) controls the cumulative probability mass from which the next token is sampled: a value of 1.0 retains the full vocabulary distribution and is the provider default, ensuring no artificial truncation of the output distribution. `max_tokens` (maximum output length) sets an upper bound on the number of tokens the model may generate in a single call; using the provider default avoids artificially truncating long structured outputs (e.g., mathematical formulations or Gurobi code blocks) that vary in length across problem instances.

Both parameters are held at their provider-specified default values throughout all experiments. This choice (i) aligns our runs with each provider's recommended baseline behaviour, (ii) eliminates an additional hyperparameter dimension that would otherwise need to be reported and tuned, and (iii) facilitates reproducibility, since any reader can replicate our calls by simply omitting these arguments from their API requests. We have added explicit inline definitions at the point of first use (Section 3.1) so that the text is self-contained.

3.1.3 Modifications

1. **Section 3.1 (L319) — MODIFY**

   Before: "*such as `top_p` and `max_tokens`*"

   After: "*such as `top_p` (nucleus sampling threshold) and `max_tokens` (maximum output length), both of which are set to their default values*"

2. **Section 4, Opening Paragraph (L415) — ADD**

   After: "*On this machine, the system is exclusively responsible for orchestrating the multi-agent workflow, executing the generated solver code within a sandbox, and managing the shared message pool, while the DeepSeek-V3-0324 model is accessed via a cloud API. For all API calls, `top_p` is set to 1.0 (i.e., no nucleus truncation) and `max_tokens` is left unspecified to default to the model's maximum output length. This configuration preserves the full output distribution and prevents the artificial truncation of lengthy structured outputs, such as mathematical formulations and solver code.*"

3.2 Comment 3.2 and 3.7 — Dual Use of Variable i

3.2.1 Comment

1. **Comment 3.2:** "In line 332, the statement 'Denote the output at step i as y_i' raises a question: Is this i the same as the i in '$A = \{a_i\}_{i=0}^{n}$,' where i represents both the iteration step count (as indicated in Figure 1) and the agent number in A?"

2. **Comment 3.7:** "In section 3.2, there is a lack of clarity in differentiating the roles of i and k. It is essential to elucidate the distinction between these two parameters."

3.2.2 Response

We deeply appreciate the reviewer's rigorous attention to detail. Pointing out the confusion between i and k is invaluable. Such meticulous feedback is indispensable for ensuring the mathematical clarity of the paper. We have carefully revised the entire text to eliminate this ambiguity. The variable i was indeed overloaded: in $\mathcal{A} = \{a_i\}_{i=0}^{n}$ it identifies agent types, while in Steps 2–4 it served as the step index. We have disambiguated by reserving i exclusively for agent identifiers and introducing k ($k = 1, 2, \ldots, K$) for step indexing. Additionally, all set-valued notation is now written in calligraphic form, so that the global agent set is denoted by $\mathcal{A}$ and the selectable agent set at step k by $\mathcal{A}_k$, and a_k denotes the agent selected at step k. The state formula has also been corrected, the definition of K has been clarified, the Step 4 update is now written as $\mathcal{A}_{k+1}$, and the backtracking description has been aligned with this notation.

For clarity, we provide an illustrative example. The full agent set is $\mathcal{A} = \{a_i\}_{i=0}^{5}$, where PM-Agent a_0 orchestrates the remaining five: DE ($i=1$), ME ($i=2$), PY ($i=3$), TE ($i=4$), BA ($i=5$). Set $K = 3$, let the initial selectable set be $\mathcal{A}_1 = \{a_1, \ldots, a_5\}$, and suppose the PM-Agent skips DE and starts directly from ME (i.e., selects a_2 at step $k=1$). Note that the step index k is determined by the position within the forward pass, whereas the agent identifier i is a fixed property of each agent; the two are in general distinct:

Step k	**Selectable set** $\mathcal{A}_k$	**Selected agent** a_k	**Output** y_k
Round 1 — Iterative Modeling (PM-Agent skips DE; $k \neq \mathrm{id}(a_k)$*)*			
1	$\{a_1, a_2, a_3, a_4, a_5\}$	a_2 (ME, $i=2$)	y_1
2	$\{a_3, a_4, a_5\}$	a_3 (PY, $i=3$)	y_2
3	$\{a_4, a_5\}$	a_4 (TE, $i=4$)	y_3
Step 7: Exception Backtracking ($H = [a_2, a_3, a_4]$*; pop in LIFO order)*			
Pop a_4 (TE): $(\delta_3, \xi_3) \leftarrow a_4(C)$; δ_3=True (propagate upward);			
revert C:=$\{y_1, y_2\}$, restore $\mathcal{A}_3$=$\{a_4, a_5\}$, H:=$[a_2, a_3]$			
Pop a_3 (PY): $(\delta_2, \xi_2) \leftarrow a_3(C)$; δ_2=False (claims fault), provides refined y_2';			
revert C:=$\{y_1, y_2'\}$, set $\mathcal{A}_{k+1}$:=$\mathcal{A}_{>\mathrm{id}(a_3)}$=$\{a_4, a_5\}$			
Round 2 — Re-Iterative Modeling (k resets; C=$\{y_1, y_2'\}$*;* $\mathcal{A}_1$=$\{a_4, a_5\}$*)*			
1	$\{a_4, a_5\}$	a_4 (TE, $i=4$)	y_1'

This example demonstrates four key distinctions: (1) agent identifier i is fixed (e.g., TE is always a_4, $i=4$), while step index k resets per forward pass; (2) the same agent (TE, $i=4$) can appear at different step indices ($k=3$ in Round 1, $k=1$ in Round 2), and conversely, the same step index ($k=1$) refers to different agents in different rounds (ME in Round 1 vs. TE in Round 2); (3) K bounds the number of selections *per forward pass*, not the cumulative total; and (4) the selectable set $\mathcal{A}_k$ shrinks monotonically within a forward pass under the sub-sequence rule, and resets to $\mathcal{A}_{>\mathrm{id}(a_3)}$=$\{a_4, a_5\}$ at the start of Round 2, ensuring the re-forward respects the same monotonic-identifier ordering anchored to the last fault-claiming agent.

3.2.3 Modifications

1. **Section 3.2, Step 1 (L303) — MODIFY**

   Before: "*the upper limit of selection attempts as K, and the predetermined set of agents as $A = \{a_i\}_{i=0}^{n}$ … to facilitate exception backtracking.*"

   After: "*the maximum number of agents to be selected per forward pass as K, and the predetermined set of agents as $\mathcal{A} = \{a_i\}_{i=0}^{n}$ … to facilitate exception backtracking. Throughout the algorithm, the subscript i is reserved exclusively for agent identifiers in $\mathcal{A}$, while k ($k = 1, 2, \ldots, K$) denotes the step index within a single forward pass.*"

2. **Section 3.2, Step 2 (L306–310) — MODIFY**

   Before: "*Denote the output at step i as y_i, and the state as $s_i = (P, C, |A| - K - i)$.*"

   After: "*Denote the output at step k as y_k, the set of selectable agents at step k as $\mathcal{A}_k$, and the state as $s_k = (P, C, K - k, \mathcal{A}_k)$.*" The text following Equation (1) now further clarifies that the global agent set is denoted by $\mathcal{A}$, $\mathcal{A}_k$ denotes the selectable set at step k, A_k denotes the random variable for the selected agent, and a_k denotes the realized selected agent.

3. **Section 3.2, Steps 2–7 (L306–335) — MODIFY**

   Change: All step-indexed subscripts i were replaced with k throughout Steps 2–7 (including y_k, s_k, a_k, f_k, $\mathcal{T}_k$, A_k, S_k, δ_k, and ξ_k). In addition, all set-valued notation is now written in calligraphic form, including the global agent set $\mathcal{A}$ and the intermediate selectable sets $\mathcal{A}_k$. The subsequence rule in Step 4 now uses $\mathrm{id}(a_k)$ instead of i and explicitly updates the next-step selectable set as $\mathcal{A}_{k+1}$. The backtracking description in Step 7 correspondingly restores $\mathcal{A}_k$ rather than the global action space. Step 5's termination condition was also simplified from "current iteration count k exceeds K" to "all K steps have been executed."

4. **Figure 1 — MODIFY (symbol update; updated figure shown in Comment 3.4)**

   Change: The two flowchart boxes that previously displayed $a_i \leftarrow a_0(P, C)$ (Agent Selection) and $y_i \leftarrow a_i(P, C)$ (Action Execution) have been updated to $a_k = \pi_{\mathrm{PM}}(s_k)$, $a_k \in \mathcal{A}_k$ and $y_k \leftarrow a_k(P, C)$ respectively, aligning Figure 1 with the revised notation in Steps 2–3 of Section 3.2. The updated figure is presented in Comment 3.4.

3.3 Comment 3.3 — Equation (1) Symbols Undefined [COMPLETED]

3.3.1 Comment

"The notation of variables in equation (1) is not clearly defined. What are the definitions of e, S_i, s, and A_i?"

3.3.2 Response

We deeply appreciate the reviewer's rigorous attention to notation. Pointing out the undefined symbols e, S_i, s, and A_i in Equation (1) is invaluable; such meticulous feedback is indispensable for ensuring the mathematical clarity and notational consistency of the paper. We agree that the original notation around Equation (1) was unclear. After re-checking the implementation, we further determined that the probabilistic formulation itself was stronger than warranted by the actual PM-Agent selection mechanism. In the revised manuscript, we therefore simplified Equation (1) to a deterministic state-dependent selection function, so the previously ambiguous symbols e, S_i, and A_i no longer appear. We then added explicit definitions for the remaining symbols in the revised formulation. For ease of reference, we summarize them below:

Symbol	Definition
π_{PM}	The PM-Agent selection function, mapping the current state to the next selected agent within the admissible set
s_k	Current state at step k: $(P, C, K-k, \mathcal{A}_k)$, comprising the problem description, message pool history, remaining selection quota, and the selectable agent set
$\mathcal{A}_k$	The set of agents available for selection at step k
a_k	The agent selected by the PM-Agent at step k
P	The natural language problem description
C	The shared message pool storing all agent outputs
K	The maximum number of agents to be selected per forward pass

3.3.3 Modifications

1. **Section 3.2 Equation (1) (L330–334) — MODIFY**

   Before: "*Denote the output at step i as y_i, and the state as $s_i = (P, C, A-K-i)$. The policy selection function is defined as...* "

   After: "*Denote the output at step k as y_k, the set of selectable agents at step k as $\mathcal{A}_k$, and the state as $s_k = (P, C, K-k, \mathcal{A}_k)$. The PM-Agent selection function is defined as:*"

   $$a_k = \pi_{\text{PM}}(s_k)\,, \quad a_k \in \mathcal{A}_k \tag{1}$$

   "*where s_k denotes the current state comprising the problem description, message pool history, remaining selection quota, and the current selectable set $\mathcal{A}_k$; π_{PM} denotes the PM-Agent selection function; $\mathcal{A}_k \subseteq \mathcal{A}$ denotes the set of agents available for selection at step k; and a_k denotes the selected agent at step k. In the current implementation, the selectable set is restricted by the sub-sequence rule and the PM-Agent is decoded deterministically; accordingly, the realized selection process is better described as a deterministic, state-dependent mapping over an ordered admissible set.*"

3.4 Comment 3.4, 3.5 and 3.6 — Variable Inconsistencies in Figure 1

3.4.1 Comment

1. **Comment 3.4:** "In Figure 1, the variable N lacks a definition. It would be advantageous to provide a detailed explanation of how N should be determined."

2. **Comment 3.5:** "In Figure 1, the exception backtracking section features a Y arrow from the 'error: a_i?' box, indicating that an error with a_i results in a_i being added to stack H. Is this interpretation accurate, or should the process be reversed?"

3. **Comment 3.6:** "In lines 345-346, the phrase 'If the current iteration count k exceeds the maximum permissible selection K,' does not reference the values of k and K in Figure 1. As noted by the author in line 319, the SOP in this section is aligned with Figure 1. It seems that several variables in this section differ from those depicted in Figure 1."

4. **Comment 3.7:** "In section 3.2, there is a lack of clarity in differentiating the roles of i and k. It is essential to elucidate the distinction between these two parameters."

3.4.2 Response

We thank the reviewer for this careful observation. In the original Figure 1, the symbol N was used to denote the maximum selection limit, whereas the main text consistently uses K for the maximum number of agents to be selected per forward pass. To eliminate this inconsistency, we redrew Figure 1 and replaced N with K throughout the flowchart. We also added an explicit sentence in Step 1 of Section 3.2 to state that the same notation K is used in Figure 1, thereby ensuring full consistency between the figure and the textual definition. In addition, following the symbol disambiguation addressed in Comments 3.2 and 3.7, the two flowchart boxes previously labelled $a_i \leftarrow a_0(P, C)$ and $y_i \leftarrow a_i(P, C)$ have been updated to $a_k = \pi_{\mathrm{PM}}(s_k)$ and $y_k \leftarrow a_k(P, C)$ respectively, consistent with the revised notation in Steps 2–3 of Section 3.2.

3.4.3 Modifications

1. **Figure 1 (L330–336) — MODIFY**

   Before: "*Figure 1 used the symbol N to denote the maximum selection limit.*"

   After: "*Figure 1 now uses K to denote the maximum selection limit, consistent with the definition in Step 1 of Section 3.2.*"

   Before: "*the exception backtracking section features a Y arrow from the 'error: a_i?' box.* "

   After: "*Figure 1 now uses Error δ_k to denote the error signal from agent a_k, and the Y arrow from the 'error?' box indicates that if δ_k is True (i.e., an error is detected), then a_k re-executes its action.*"

   Before: " *If the current iteration count k exceeds the maximum permissible selections K, emit the message pool C and the execution result.*"

   After: "*Figure 1, Previously, we used i; now we uniformly use k.*"

3.5 Comment 3.8 and 3.9 — Figure 2 Shows Wrong Problem (Airplane Landing, not ECR)

3.5.1 Comment

1. **Comment 3.8:** "In lines 420-432 of section 3.4, the authors aim to describe the SOP-MAS process for a classic ECR case in container SCM using Figure 2. However, the figure actually represents the Aircraft Landing Problem. Additionally, the terms PD and MA agents are used instead of the previously established PM and BA."

2. **Comment 3.9:** "Considering that Figure 2 is a crucial example of the framework, a more comprehensive explanation of the procedure illustrated in the figure would be beneficial and would greatly enhance readers' understanding. For instance, the exception backtracking component could be discussed in more depth within the content."

3.5.2 Response

We sincerely thank the reviewer for identifying this error. We acknowledge that Figure 2 inadvertently illustrated the Aircraft Landing Problem from the ComplexOR benchmark rather than the intended ECR case. This was a careless oversight during figure preparation. To produce an authentic illustration, we re-ran the SOP-MAS framework on a single-period, three-port ECR instance (ports A, B, C with supply $[100, 50, 0]$ and demand $[0, 0, 150]$ TEU) using the full five-agent configuration (K=5, with `is_backward=True`). The system successfully solved the

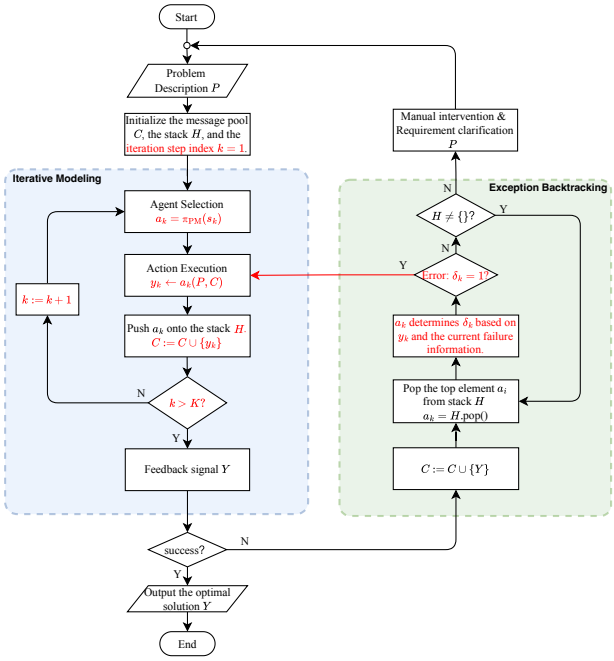

Figure 2: Updated Figure 1 included for reviewer reference.

problem, obtaining an optimal objective value of ¥650, and all five agent outputs (DE-Agent through BA-Agent) were recorded. Figure 2 has been replaced with a new figure based on these actual outputs, with agent labels corrected to PM-Agent, DE-Agent, ME-Agent, PY-Agent, TE-Agent, and BA-Agent. Additionally, as suggested in Comment 3.9, we have expanded the explanation to include a dedicated paragraph describing the exception backtracking mechanism within the illustrative example.

3.5.3 Modifications

1. **Section 3.4 (Figure 2) — REPLACE**

   Before: "*Figure 2 illustrated the Aircraft Landing Problem with incorrect agent labels (PD, MA).*"

   After: "*Figure 2 has been replaced with a correct ECR example showing the full SOP-MAS workflow on a three-port empty container repositioning problem, with agent labels corrected to PM-Agent, DE-Agent, ME-Agent, PY-Agent, TE-Agent, and BA-Agent.*"

2. **Section 3.4 (Figure caption) — MODIFY**

   Before: "*SOP-MAS Workflow Example*"

   After: "*End-to-end SOP-MAS workflow on an ECR instance (3 ports, T=1): from natural-language description to optimal repositioning plan*"

3. **Section 3.4 (opening paragraph) — REWRITE**

   Before: "*In order to effectively demonstrate the capabilities of the proposed SOP-MAS workflow...a step-by-step exposition based on a classic case of ECR in container SCM is provided, as illustrated in Figure 2.*"

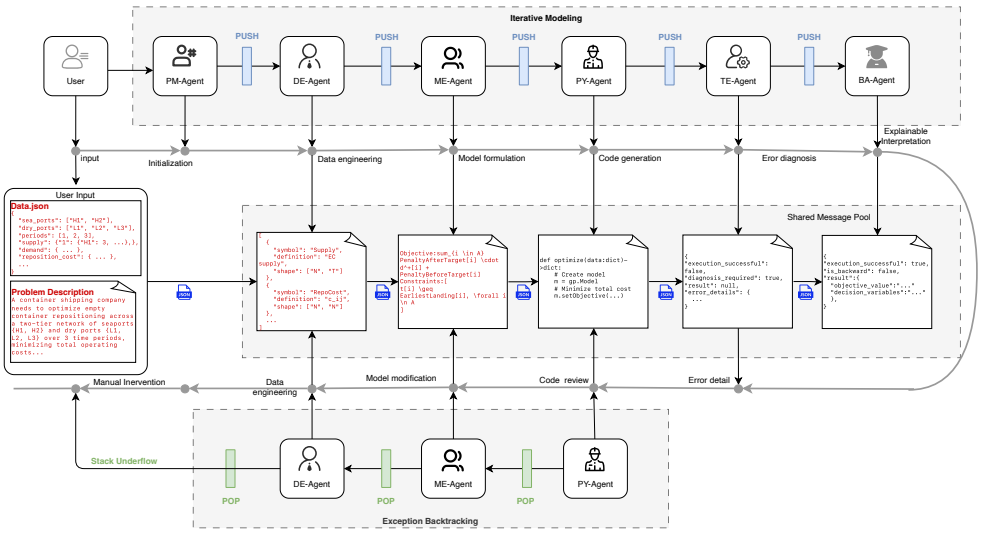

Figure 3: Updated Figure 2.

After: "*To demonstrate the end-to-end capability of the proposed SOP-MAS framework, we apply it to a single-period, three-port ECR instance. Ports A and B hold surplus containers (100 and 50 TEU, respectively) while port C demands 150 TEU; the unit transportation costs c_{ij} range from 2 to 10 ¥/TEU. The SOP-MAS is configured with K=5 collaborating agents, backtracking enabled, DeepSeek-V3-0324 as the backbone LLM, and Gurobi as the solver. The complete workflow is illustrated in Figure 2.*"

4. **Section 3.4 (agent walkthrough paragraph) — REWRITE**

Before: "*The DE-Agent converts unstructured data and business constraints into consistent and reproducible modeling parameters... The ME-Agent automatically formulates a multi-period, multi-node mathematical model... The PY-Agent generates comprehensive, executable Python code... The TE-Agent executes the code within a sandboxed environment... The BA-Agent translates the model outputs...into an interpretable business-analysis report.*"

After: "*The PM-Agent orchestrates the five agents in sequence. First, the DE-Agent parses the natural-language problem description and extracts structured parameters: the supply vector S=[100, 50, 0], the demand vector D=[0, 0, 150], and the cost matrix c_{ij}. Second, the ME-Agent formulates the LP: $\min \sum_{i,j} c_{ij} x_{ij}$ subject to flow-balance constraints $\sum_j x_{ij} - \sum_j x_{ji} = S_i - D_i$ for each port i and non-negativity $x_{ij} \geq 0$. Third, the PY-Agent generates executable Gurobi code implementing this model. Fourth, because the TE-Agent is selected in this example, it performs one supplementary execution during the forward pass, captures the solver log, and produces a structured diagnostic report; Gurobi solves the resulting LP in 2 iterations, yielding an optimal total repositioning cost of ¥650. After the forward pass, the framework enters its unified verification stage and directly reuses this successful TE-Agent result. Fifth, the BA-Agent translates the optimal solution (x_{AB}=100, x_{BC}=150, with all other flows equal to zero) into a business-analysis report, correctly stating the optimal total transportation cost of CNY 650, explaining why the low-cost A–B and B–C arcs are selected to satisfy port C's demand through Port B as the transfer hub, and clarifying why the expensive direct A–C route and reverse movements remain zero.*"

5. **Section 3.4 (exception backtracking paragraph) — ADD**

   After: "*Should an execution error or incorrect solution occur, the SOP-MAS framework triggers the exception backtracking mechanism. For instance, had the PY-Agent generated code that omitted a flow-balance constraint, the unified verification stage would detect the resulting failure. If the TE-Agent had been selected in that forward pass, its earlier execution report in the shared message pool would provide additional diagnostic evidence; otherwise, diagnosis would rely on the framework-level failure information alone. A stack-based reverse traversal is then performed: agents are popped in last-in-first-out order and each is queried to determine whether the error originated from its output. Upon locating the responsible agent, that agent provides a refined result, which is pushed back onto the stack, and the forward modeling resumes from the corrected step onward using the remaining collaboration rounds. This iterative correction loop continues until a valid solution is obtained or all collaboration rounds are exhausted.*"

3.6 Comment 3.10 — Accuracy Increase Value Inconsistency

3.6.1 Comment

"In line 501, the assertion '...realizing an increase in accuracy by 8.89% over SPM' is inconsistent with the accuracy rates for COT and SPM, which are 52.26% and 48.08%, respectively. Consequently, the increase is not 8.69%."

3.6.2 Response

We verified the calculation: $(52.26 - 48.08)/48.08 = 8.69\%$, confirming the relative improvement is correct. To eliminate ambiguity between percentage-point differences and relative improvement, we revised the phrasing to explicitly state both values. During this revision, we conducted a self-audit of the entire manuscript and identified the same ambiguity pattern in Section 4.3 (ablation study), which has been corrected accordingly.

3.6.3 Modifications

1. **Section 4.1 (L423) — MODIFY**

   Before: "*realizing an increase in accuracy by 8.69% over SPM*"

   After: "*realizing an increase in accuracy from 48.08% to 52.26% (a relative improvement of 8.69%) over SPM*"

2. **Section 4.3 (L484) — MODIFY**

   Before: "*there is a 28.75% reduction in accuracy compared to the control group*"

   After: "*there is a decrease in accuracy from 86.59% to 57.84% compared to the control group*"

3. **Section 4.3 (L484) — MODIFY**

   Before: "*a 13.42% reduction in accuracy relative to the control group*"

   After: "*a decrease in accuracy from 86.59% to 73.17% relative to the control group*"

3.7 Comment 3.11 — `is_backward` Formatting Inconsistency

3.7.1 Comment

"In line 534, the phrase '...was deactivated (is_backward = False)' should have 'is_backward = False' emphasized in bold, as it is in line 369."

3.7.2 Response

We unified the formatting of `is_backward` throughout the manuscript to use consistent monospace type.

3.7.3 Modifications

1. **Section 4.2 (L431) — MODIFY**

   Before: "*is_backward=False*"

   After: "*`is_backward = False`*"

3.8 Comment 3.12 — Symbols "+" and "-" Defined but Unused

3.8.1 Comment

"In lines 581-582, the definitions of the symbols '+' and '-' are provided but are not applied anywhere in the manuscript."

3.8.2 Response

We thank the reviewer for this careful observation. The reviewer is correct: the "+" and "−" symbols were defined to indicate whether backtracking is enabled in each ablation group, yet they were never actually used. We have now incorporated these annotations directly into the group labels of Table 2, so readers can immediately identify each group's backtracking status without cross-referencing a separate notation paragraph, making the table more self-contained.

3.8.3 Modifications

1. **Table 2 (L473–478) — MODIFY**

   Before: "*Group1, Group2, Group3, Group4, Group5, Control Group*"

   After: "*Group1 (−), Group2 (+), Group3 (+), Group4 (+), Group5 (−), Control Group (+)*"

3.9 Comment 3.13 — S_i^t Variable t Undefined

3.9.1 Comment

"In line 689, the variable 't' within 'S_i^t' lacks a definition."

3.9.2 Response

We thank the reviewer for identifying this omission. The time period index t was indeed used without prior definition in the problem description. We have added an explicit definition of the time period set T and index $t \in T$ at the beginning of Section 5.2. Furthermore, upon systematic inspection prompted by this comment, we discovered that the ECR model notation (Sections 5.2 and Appendix C) shared several symbols with the SOP-MAS methodology (Section 3), including H (backtracking stack vs. seaport set), K/k (agent iteration steps vs. transport modes), S (state

variable vs. supply), and y (agent output vs. rental quantity). To eliminate all ambiguities, we have comprehensively redesigned the ECR notation using a three-tier convention (calligraphic capitals for sets, uppercase Romans for parameters, lowercase Romans for decision variables) while keeping Section 3 unchanged. The complete mapping is provided in the Modifications subsection below.

3.9.3 Modifications

1. **Section 5.2, Problem Description (line 536) — ADD**

   After: "*such that* $\mathcal{N} = \mathcal{P} \cup \mathcal{L}$. *Let* $T = \{1, 2, \ldots, |T|\}$ *denote the set of time periods in the planning horizon, indexed by* $t \in T$. *The collection of edge* E *symbolizes*"

2. **Section 5.2 + Appendix C — MODIFY (Notation Harmonization)**

   To eliminate all symbol conflicts with the SOP-MAS methodology (Section 3), the ECR model notation has been systematically redesigned as follows:

Old Symbol	New Symbol	Meaning
H	$\mathcal{P}$	Seaport set
L	$\mathcal{L}$	Dry port set
$N = H \cup L$	$\mathcal{N} = \mathcal{P} \cup \mathcal{L}$	Node set
K	$\mathcal{M}$	Transport mode set
k	m	Transport mode index
S_i^t	Q_i^t	Empty container supply
y_i^t	r_i^t	Rental quantity

3.10 Comment 3.14 — ECR Case Solution Lacks Numerical Evidence

3.10.1 Comment

1. **Comment 3.14:** "In lines 719-729, the solution derived from the SOP-MAS framework is presented, yet the discussion lacks supporting numerical evidence. Including a more comprehensive discussion with examples would significantly enhance the paper's quality."

2. **Comment 3.15:** "In line 745, the mention of the reduction in repositioning cost as a percentage appears abruptly in the conclusion. This improvement should be addressed in Section 5.3, where the model's solution is analyzed."

3.10.2 Response

We thank the reviewer for this constructive suggestion. The original Section 5.3 contained only qualitative descriptions without specific numerical results. We have substantially revised this section by (1) reporting the optimal objective value and cost breakdown for Scenario 1 (sea–land coordination), (2) adding a detailed cost comparison table between the two scenarios with percentage changes, and (3) providing concrete numerical evidence for each qualitative claim. Additionally, we corrected a misstatement in the Conclusion: the 12.18% reduction refers to total operational costs, not repositioning costs alone.

3.10.3 Modifications

1. **Section 5.3, Results Analysis (lines 548–552) — MODIFY**

Before: "*To evaluate the benefits of the sea–land intermodal transport mode, a comparative analysis was conducted under the scenario wherein dry ports are unable to facilitate the exchange of empty containers among themselves, as illustrated in Figure 5. The results show that the repositioning of empty containers from dispersed dry ports to seaports experiencing shortages becomes increasingly challenging. Consequently, there is an increase in both the volume of empty containers transported directly from dry ports to seaports and the volume circulated among seaports, ultimately leading to elevated overall costs. Furthermore, the exacerbated difficulty in empty container flows leads to a greater number of containers failing to arrive at shortage seaports in a timely manner, which in turn increases the leasing demand at the affected seaports and significantly escalates repositioning costs.*"

After: "*Under the sea–land coordination scenario (Scenario 1), wherein hinterland lateral repositioning is permitted, the optimal total cost over the five-period planning horizon is CNY 1,190,336, comprising repositioning costs of CNY 763,966, holding costs of CNY 30,870, and leasing costs of CNY 395,500, as summarized in Table 6.* To evaluate the benefits of hinterland lateral repositioning, a comparative analysis was conducted under the scenario without inter-dry-port exchange (Scenario 2), as illustrated in Figure 5. As shown in Table 6, the total cost increases from CNY 1,190,336 to CNY 1,355,388 (+13.86%). This increase is primarily driven by the repositioning costs, which rise from CNY 763,966 to CNY 925,238 (+21.11%). Within the repositioning category, the sea–land repositioning cost surges from CNY 594,952 to CNY 864,910 (+45.38%), as the network must compensate for the loss of hinterland lateral flows (CNY 110,718 reduced to zero) by routing more containers through the sea–dry corridors. Furthermore, the exacerbated difficulty in empty container flows leads to a greater number of containers failing to arrive at shortage seaports in a timely manner, which in turn increases the leasing demand at the affected seaports from CNY 395,500 to CNY 406,000 (+2.66%). Overall, enabling hinterland lateral repositioning yields a 12.18% reduction in total operational costs."

2. **Section 5.3 — ADD (New Table)**

   After: A new table (Table: Cost comparison between the two scenarios) has been inserted in Section 5.3, presenting a seven-row cost breakdown comparing Scenario 1 and Scenario 2 across repositioning sub-categories, holding, leasing, and total costs, with percentage changes for each row.

3. **Section 6, Conclusions (line 571) — MODIFY**

   Before: "*leading to a 12.18% reduction in repositioning costs for the shipping company.*"

   After: "*leading to a 12.18% reduction in total operational costs for the shipping company.*"

3.11 Comment 3.16 — Future Research Directions in Conclusion

3.11.1 Comment

"In the concluding section, it would be advantageous to explore potential future research directions. Given the rapid development and interest in this area of operations research, such a discussion would be a valuable addition."

3.11.2 Response

We thank the reviewer for this constructive suggestion. We have restructured the concluding section to include a dedicated discussion of future research directions, covering three concrete avenues: (1) extending the SOP-MAS framework to more complex optimization paradigms such as stochastic and robust optimization; (2) deploying the framework in interactive, human-in-the-loop settings that allow practitioners to intervene at agent handoff points; and (3) developing comprehensive evaluation metrics for the BA-Agent, whose qualitative business interpretations fall outside the scope of the current CER/RER/AR solution-quality metrics.

3.11.3 Modifications

1. **Section Conclusions (line 597) — MODIFY**

   Before: "*As training parameters and scale increase, pre-trained LLMs are anticipated to achieve further advancements in capability. By integrating general-purpose LLMs with customized knowledge bases, the cost associated with decision-making can be significantly reduced. From the perspective of shipping companies, which occupy a central role in the empty container repositioning process, the proposed approach can be further extended to support additional stakeholders, including seaports, dry ports/inland container yards, shippers/customers, and third-party leasing companies.*"

   After: "*Several promising directions merit further investigation. First, extending the SOP-MAS framework to more complex optimization paradigms—such as stochastic programming, robust optimization, and multi-objective optimization—would broaden its applicability to real-world ECR scenarios involving demand uncertainty, disruption risks, and conflicting stakeholder objectives. Second, deploying the framework in interactive, human-in-the-loop settings would allow logistics practitioners to intervene at agent handoff points (e.g., validating the OR Specialist's mathematical model before it is passed to the Python Developer), enabling iterative refinement and supporting a broader set of stakeholders, including seaports, dry ports, shippers, and third-party leasing companies, within the container supply chain. Third, while the CER, RER, and AR metrics rigorously evaluate the optimization core of SOP-MAS, the BA-Agent introduces a qualitatively different output type—business interpretation and decision recommendations—that extends beyond solution correctness. Developing dedicated evaluation dimensions for this agent, such as interpretation accuracy, recommendation completeness, and decision actionability, and validating them through structured expert studies with logistics practitioners, would enrich the multi-agent evaluation framework and provide rigorous empirical evidence of the BA-Agent's contribution to decision support.*"

3.12 Comment 3.17 — Move Math Model from Appendix to Main Text

3.12.1 Comment

"A minor suggestion: the authors might consider integrating the mathematical programming model for the problem into the main body of the paper, rather than relegating the LaTeX version to the appendix. While it is understandable that the authors aim to highlight the results from the LLM and MAS approach, this presentation may complicate the understanding of the problem and model. However, it is understandable if the authors choose to retain the model in its current LaTeX format."

3.12.2 Response

We thank the reviewer for this constructive suggestion. We carefully considered whether to move the full mathematical programming model into the main text. Since the formulation is relatively lengthy and the main contribution of the paper is the SOP-MAS framework and its empirical decision-support process, we decided to retain the complete model in Appendix C. This choice is also consistent with the reviewer's note that retaining the current LaTeX-format model is understandable.

To address the reviewer's readability concern without interrupting the flow of the empirical analysis, we strengthened the signposting in the main text. Specifically, Section 5.2 now explicitly states that the problem setting, key decision logic, variables, and operational assumptions are summarized in the main text, while the complete mathematical formulation is provided in Appendix C. We also revised the Results Analysis transition and the Appendix C title/introductory sentence to make the appendix's role clearer as the complete machine-generated mathematical model used in the empirical study.

3.12.3 Modifications

1. **Section 5.2, Problem Description — MODIFY (Strengthen signposting to Appendix C)**

   Added: "*To keep the main text focused on the problem setting and empirical analysis, the complete mathematical formulation used in the ECR case is provided in Appendix C, while the key decision logic, variables, and operational assumptions are summarized above.*"

2. **Section 5.3, Results Analysis — MODIFY**

   Before: "*Drawing upon the ECR problem described in Section 5.2 and the data presented in Section 5.1, the SOP-MAS framework was employed for end-to-end optimization, with the mathematical model detailed in Appendix C.*"

   After: "*Drawing upon the ECR problem described in Section 5.2 and the data presented in Section 5.1, the SOP-MAS framework was employed for end-to-end optimization. The complete machine-generated LaTeX listing of the mathematical model is retained in Appendix C.*"

3. **Appendix C title and introductory sentence — MODIFY**

   Before: "*ECR Mathematical Model (LaTex). Since one of our core contributions is the automatic synthesis of publication ready modeling code, we included raw LaTeX snippets primarily to illustrate the output format generated by the SOP-MAS framework.*"

   After: "*ECR Mathematical Model (LaTeX Listing). Since one of our core contributions is the automatic synthesis of publication-ready modeling code, we retain the raw LaTeX listing below to illustrate the complete mathematical formulation generated by the SOP-MAS framework. The main text summarizes the problem setting, key decision logic, and operational assumptions in Section 5.2, while this appendix provides the full formulation for readers seeking model-level detail.*"